

PROVBIND: Continuous Provenance-Bound Runtime Integrity Verification for Detecting Software Supply Chain Attacks in Cloud-Native Containers

Takorn Sripetcharakul, Krittakorn Saetia, Sirasit Wongtawewat, Panachai Buddharaksa, Somchart Fugkeaw
Sirindhorn International Institute of Technology, Thammasat University, Thailand
6622771283@g.siit.tu.ac.th, 6622770475@g.siit.tu.ac.th, 6622770426@g.siit.tu.ac.th,
6622780664@g.siit.tu.ac.th, somchart@siit.tu.ac.th

Abstract—Software supply chain attacks reach production through legitimate and fully authorised channels, and container images have become a principal vehicle for their delivery. Existing defences establish trust before execution: build frameworks emit signed provenance and a Software Bill of Materials (SBOM), and admission controllers verify these signatures before a workload is permitted to start. Such mechanisms prove the origin of an artifact rather than its trustworthiness, and they establish trust at a single instant. Runtime monitors observe behaviour thereafter, but derive their notion of correct behaviour from observed execution and remain blind to the signed declaration that already describes what the container was supposed to contain. In this paper we present PROVBIND, a framework that retains the attestation verified at admission and compiles it into an expected behavioural envelope comprising the declared file set with per-layer attribution, the package dependency closure annotated by depth, and the process closure implied by the declared entrypoint; a gradient-boosted model supplies capability and egress expectations where the declaration is silent. An eBPF collector observes process, file, and network events at kernel level, and a binding verifier determines for each event whether it contradicts the declared envelope. Detection is thereby reframed from an unsupervised statistical problem into a verification problem grounded in cryptographic evidence: every alert names the specific signed claim it contradicts, is ranked by the offending component’s distance from the declared dependency closure, and is attributed to the image layer that introduced it. A separate re-evaluation loop withdraws trust from already-running workloads when signing key state or external revocation intelligence changes. Detection results are stored in a graph database supporting layer attribution and multi-hop deviation correlation.

Index Terms—software supply chain security, container security, runtime verification, provenance attestation, eBPF, SBOM

I. INTRODUCTION

Cloud-native applications increasingly rely on complex software supply chains comprising open-source packages, transitive dependencies, base container images, build tools, registries, and CI/CD pipelines. Kubernetes further amplifies this dependency by enabling containerized workloads assembled from these components to be deployed at scale. However, an adversary that compromises a dependency, build process,

repository, or container image can propagate malicious code through the same trusted channels used by legitimate software [1]–[3], [27]. Consequently, a workload may reach a Kubernetes cluster as an apparently legitimate container even though an upstream component has already been compromised.

Modern supply-chain defenses therefore focus on establishing artifact provenance before deployment. *in-toto* provides verifiable evidence of software production steps [4], SLSA defines progressively stronger build-provenance guarantees [5], and Sigstore provides practical software-signing and transparency infrastructure [6]. Software Bills of Materials (SBOMs) complement these mechanisms by describing the components and dependencies within an artifact, although adoption, completeness, and correctness remain challenges [11]–[13]. Together, these mechanisms establish valuable evidence about what an artifact contains, where it originated, and how it was produced.

In Kubernetes, such evidence can be verified before workload admission. An admission policy can validate image signatures, provenance, builder identity, SBOM information, and image digests before allowing a container to execute. However, this creates a critical *temporal and semantic gap*: successful admission establishes artifact identity and provenance at one point in time, but does not establish whether subsequent runtime behavior remains consistent with the evidence that justified admission. Once execution begins, the verified provenance and SBOM are typically no longer involved in runtime security decisions.

Consider a compromised transitive package embedded in a properly built and signed container image. Its image digest and signature may be valid, the provenance may correctly identify the legitimate build pipeline, and the compromised package may even appear correctly in the SBOM. Admission therefore reveals no cryptographic inconsistency. The compromise becomes visible only during execution, for example when an unexpected executable is launched, a declared binary is modified, an unanticipated library is loaded, or a component initiates suspicious external communication. The problem is

therefore not necessarily incorrect supply-chain evidence, but the failure to use that evidence continuously after deployment.

Runtime container defenses address this problem from another direction. System-call confinement and dynamic filtering reduce container attack surfaces [14], [15], while runtime and eBPF-based monitoring detect malicious container behavior [16], [17], [24], [25]. Provenance-based intrusion detection further reconstructs relationships among processes, files, and network activities to detect and investigate multi-stage attacks [7]–[9]. However, expected behavior is generally derived from static analysis, previous executions, predefined policies, or learned distributions. These approaches can therefore flag unseen legitimate behavior or miss malicious behavior that remains sufficiently close to an accepted baseline, including mimicry attacks [19].

This separation exposes an important gap. Build-time mechanisms produce cryptographically bound evidence describing the admitted artifact, whereas runtime mechanisms observe what the artifact actually does, yet the two contexts remain largely disconnected. Supply-chain mechanisms establish *artifact trust before execution*, while runtime mechanisms infer *behavioral trust during execution*. Consequently, runtime alerts generally cannot determine *which property of the signed software declaration an observed event contradicts*.

Bridging this gap presents four challenges. First, **provenance and runtime events have different semantics**: provenance describes images, layers, files, packages, and dependencies, whereas runtime monitoring observes processes, file operations, capabilities, and network activities. Second, **signed declarations are behaviorally incomplete**; for example, an SBOM identifies a package but does not specify all capabilities or network destinations it requires. Third, **verification must remain efficient at runtime**, where Kubernetes nodes may generate large volumes of kernel events. Finally, **artifact trust changes over time**: signing keys, builders, or included components may become revoked or compromised after a workload has already been admitted.

To address these challenges, we propose **PROVBIND**, a framework for *continuous provenance-bound runtime integrity verification*. Rather than treating supply-chain evidence as admission-only metadata, PROVBIND retains verified provenance and SBOM information and compiles them into an indexed runtime integrity specification. The specification captures the declared file set with image-layer attribution, package dependency closure with dependency depth, and process closure derived from the declared entrypoint. Where signed evidence does not specify runtime requirements, PROVBIND supplements it with explicitly distinguished inferred capability and egress expectations.

During execution, an eBPF-based collector observes process, file, and network activities and binds them to the corresponding admitted image. PROVBIND verifies these events against the compiled specification, so a deviation can identify a concrete inconsistency with authenticated supply-chain evidence rather than merely an unexplained statistical anomaly. Dependency and image-layer information further

support deviation ranking and attribution, while residual analysis screens unusual behavior that remains within the declared envelope. In parallel, a trust re-evaluation loop reassesses running workloads when signing-key, builder, component, or revocation status changes.

The main contributions of this work are as follows:

- **Provenance-to-runtime specification compilation.** PROVBIND carries verified supply-chain evidence beyond Kubernetes admission and compiles image, provenance, and SBOM information into an indexed runtime specification, while explicitly separating declaration-derived properties from inferred expectations.
- **Continuous provenance-bound runtime verification.** An eBPF-assisted pipeline binds kernel-level runtime events to the admitted image and verifies them against its compiled specification, transforming a class of runtime detections from statistical anomaly identification into consistency verification against authenticated supply-chain evidence.
- **Dependency-aware deviation attribution.** Detected deviations are associated with the corresponding component, dependency depth, image layer, and violated specification clause, supporting provenance-aware interpretation and root-cause analysis.
- **Continuous post-admission trust re-evaluation.** An independent mechanism reassesses running workloads when the trust state of signing keys, builders, or declared components changes, extending supply-chain trust beyond the admission instant.

II. RELATED WORK

Existing work related to PROVBIND can be broadly grouped into three directions: software supply-chain integrity and attestation, container runtime security, and provenance-based intrusion detection. These directions protect complementary stages of the container lifecycle but leave a gap between the supply-chain evidence established before deployment and the behavior observed after execution begins.

A. Software Supply-Chain Integrity and Attestation

Software supply-chain attacks exploit trusted relationships among developers, dependencies, build systems, registries, and downstream consumers [1]–[3], [27]. Existing defenses therefore emphasize protecting artifact production and establishing verifiable evidence before deployment. in-toto provides end-to-end verification of software production steps [4], SLSA defines progressive levels of build-provenance assurance [5], and Sigstore provides practical software-signing and transparency infrastructure [6]. However, software-signing practices remain uneven across public package registries [10].

SBOMs complement provenance and signing by describing the components and dependencies contained in an artifact. Empirical studies have identified continuing challenges in SBOM adoption, completeness, and practical use [11], [12], while metadata-based generation can produce inconsistent component information [13]. Nevertheless, when authenticated and

bound to an image, provenance and SBOMs provide valuable evidence about artifact origin, composition, dependencies, and build history.

These mechanisms primarily establish trust *before execution*. They can support a Kubernetes admission decision by verifying that an image satisfies required supply-chain properties, but do not continuously determine whether the admitted workload subsequently behaves consistently with that evidence. PROVBIND complements these mechanisms by retaining authenticated provenance and SBOM information after admission and transforming it into a runtime-verification reference.

B. Container Runtime Security

Container-security research addresses threats that emerge after deployment through attack-surface reduction, policy enforcement, and behavioral monitoring. Confine derives system-call policies through static analysis of container images [14], whereas Optimus applies dynamic system-call filtering to reduce the exposed attack surface [15]. Broader analysis of Docker security has demonstrated the diversity of container attack and defense mechanisms [16]. Recent approaches increasingly exploit eBPF and runtime telemetry for malicious-container detection and real-time protection [17], [24], while SST-LOF applies behavioral anomaly detection to container workloads [25]. Kubernetes-specific studies further address third-party application takeover risks [20], lateral-movement mitigation [23], and cluster-level intrusion detection [30].

These approaches provide valuable runtime visibility, but their notion of expected behavior is generally obtained from static analysis, predefined policies, previously observed executions, or learned behavioral distributions. Consequently, a detected event indicates a policy violation or deviation from an expected profile, but is not necessarily linked to the authenticated supply-chain evidence that justified deployment of the container. PROVBIND instead uses the admitted image, signed provenance, and SBOM as its primary runtime reference, while learned expectations are used only for properties that the signed declarations do not express.

C. Provenance-Based Intrusion Detection

System-level provenance provides another foundation for runtime security by capturing causal relationships among processes, files, and other system objects. NodLink performs online fine-grained detection and investigation of multi-stage attacks using provenance information [7], while ORTHRUS addresses attribution in provenance-based intrusion detection [8]. Comparative analysis of state-of-the-art provenance-based intrusion detection systems shows that increased model complexity does not necessarily provide corresponding improvements in detection effectiveness [9]. Moreover, mimicry attacks demonstrate that adversaries may evade provenance-based detectors by keeping malicious activities sufficiently similar to expected behavioral patterns [19].

Overall, the three research directions protect complementary aspects of cloud-native software security. Supply-chain

mechanisms establish artifact origin and composition before deployment [4]–[6], container defenses monitor or constrain execution [14], [15], [17], [24], [25], and provenance-based detectors reconstruct runtime activity to identify attacks [7]–[9]. However, *build-time trust and runtime trust remain largely disconnected*. PROVBIND bridges this gap by carrying authenticated supply-chain evidence beyond Kubernetes admission and compiling it into an indexed runtime specification. Runtime events can therefore be associated with the authenticated declaration they contradict and attributed to the corresponding component or image layer, while an independent re-evaluation mechanism updates the trust of already-running workloads when the underlying supply-chain state changes.

III. OUR PROPOSED PROVBIND SYSTEM

A. System Model

PROVBIND provides continuous provenance-bound integrity verification for containerized workloads in Kubernetes. Its central principle is to retain authenticated software supply-chain evidence beyond admission and use it as a reference for runtime verification. Fig. 1 illustrates the architecture and interactions among seven main entities: the *Software Producer (SP)*, *Supply-Chain Repository (SCR)*, *Admission and Envelope Manager (AEM)*, *Envelope and Provenance Graph Store (EPGS)*, *Kubernetes Worker Node (KWN)*, *Runtime Integrity Verifier (RIV)*, and *Trust Re-evaluator (TR)*.

1) Software Producer (SP). The SP represents the development and CI/CD environment responsible for building and signing container images. It consumes the source repository, Dockerfile, base image, and dependency manifest and produces the container image together with its SBOM and provenance evidence. The resulting artifacts are cryptographically bound to the immutable image digest before publication.

2) Supply-Chain Repository (SCR). The SCR maintains the container artifacts and authenticated supply-chain evidence produced by the SP. It comprises the image registry, transparency log, and key-management service, which collectively provide the artifact, signing evidence, and trust-state information required for admission and subsequent trust re-evaluation.

3) Admission and Envelope Manager (AEM). The AEM establishes the runtime reference for an incoming workload. It verifies the signed image, retrieves the associated SBOM and provenance, and confirms their binding to the admitted image digest. After successful verification, the envelope compiler transforms the authenticated evidence into a *provenance-bound runtime envelope* covering file and image-layer information, package dependencies, and process and loadable-code expectations. Runtime properties not explicitly represented by signed evidence, such as capability and egress expectations, are separately inferred and marked as such.

4) Envelope and Provenance Graph Store (EPGS). The EPGS maintains the compiled runtime envelopes and their associated provenance relationships. The indexed envelope

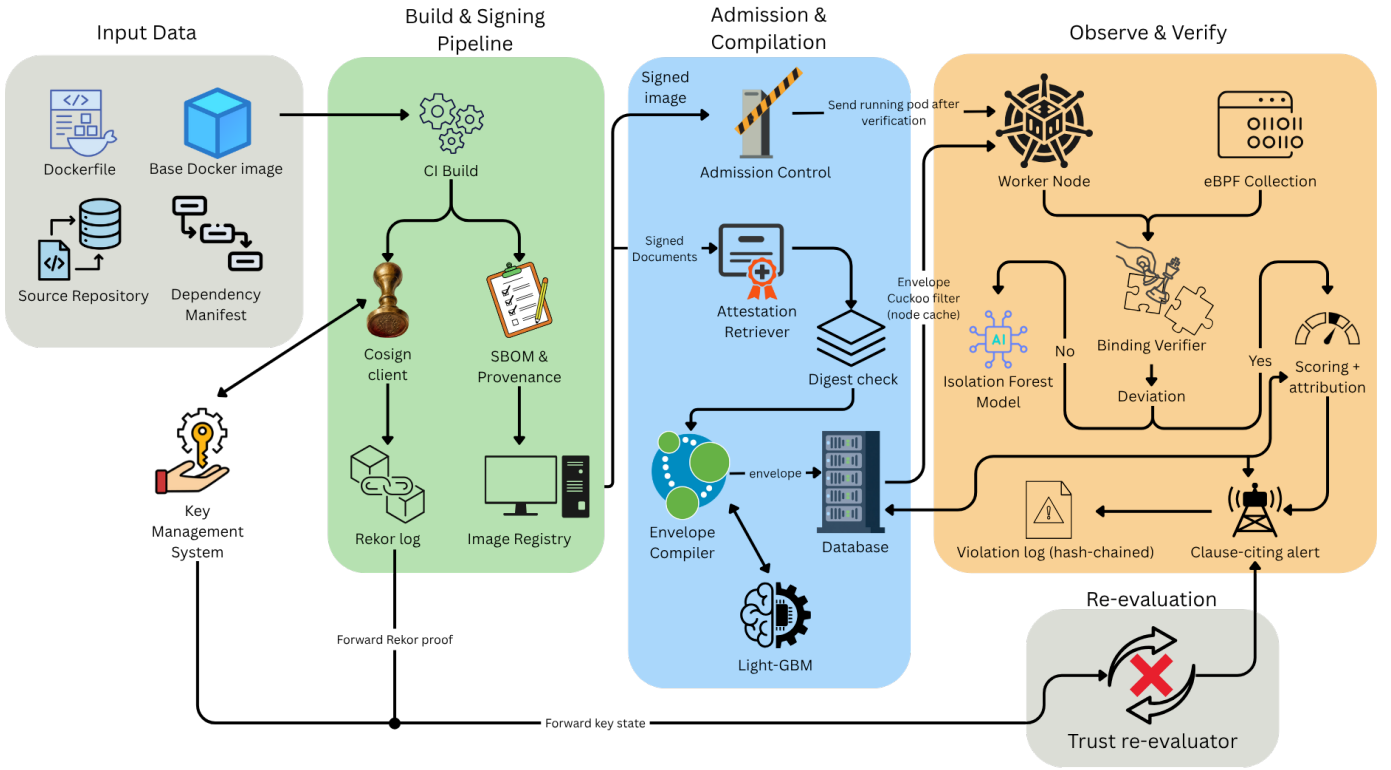


Fig. 1. System model of PROVBIND. Authenticated software supply-chain evidence is generated during the build process, verified at admission, and compiled into a provenance-bound runtime envelope. Runtime events are continuously verified against the envelope, while provenance relationships support deviation scoring and attribution. An independent trust re-evaluation path reassesses running workloads as their underlying supply-chain trust changes.

supports efficient runtime lookup, while the graph representation links container images, image layers, packages, files, processes, and detected violations. These relationships enable provenance-aware attribution, dependency tracing, and multi-hop correlation during runtime investigation.

5) Kubernetes Worker Node (KWN). The KWN executes workloads that have successfully passed admission. A kernel-level eBPF collector observes security-relevant process, file, capability, and network activities and associates each event with its container and immutable image identity. This binding enables each runtime event to be evaluated against the correct envelope maintained by the EPGS.

6) Runtime Integrity Verifier (RIV). The RIV continuously verifies events collected from the KWN against the corresponding provenance-bound runtime envelope. The Binding Verifier first identifies direct contradictions with declaration-derived or inferred expectations. Events that satisfy the envelope may undergo residual behavioral screening to detect unusual but specification-conforming activity. Detected deviations are then scored and attributed using the provenance context maintained by the EPGS, including the associated component, dependency position, image layer, and violated envelope clause. Clause-citing alerts and hash-chained violation records support subsequent investigation and audit.

7) Trust Re-evaluator (TR). The TR addresses changes in supply-chain trust after admission. It periodically reassesses

running images using signing-key status, transparency evidence, builder status, component intelligence, and relevant revocation information. If the evidence underlying an admitted workload becomes invalid or untrusted, the TR generates a trust-related alert independently of runtime behavioral deviations.

B. Threat Model

PROVBIND considers software supply-chain adversaries that compromise software before or during container construction, as well as attacks that become observable only after deployment. An adversary may compromise a direct or transitive dependency, base-image component, source artifact, or other software incorporated into an otherwise legitimate container. Consequently, a malicious artifact may be correctly built, signed, represented in the SBOM and provenance, and successfully pass Kubernetes admission without violating the cryptographic validity of its supply-chain evidence.

After deployment, the adversary may attempt to execute undeclared or modified code, load unexpected executable content, modify image-declared files, exercise unexpected privileges or capabilities, or establish network communication inconsistent with the admitted runtime envelope. The adversary may also attempt to conceal malicious activity within apparently legitimate runtime behavior. PROVBIND therefore combines direct envelope-consistency verification with residual behavioral screening.

PROVBIND further considers *post-admission trust changes*. An image that was legitimately admitted may subsequently become untrusted because, for example, its signing key or builder is revoked or a declared component is later identified as compromised. Such changes do not necessarily produce an immediate runtime deviation and are therefore handled independently through continuous trust re-evaluation.

The Kubernetes control plane, PROVBIND verification components, eBPF monitoring infrastructure, key-management service, and cryptographic primitives are assumed trusted. The underlying signing and transparency mechanisms are assumed to correctly authenticate and preserve their records. Denial-of-service attacks, compromise of these trusted components, and attacks that produce neither an observable runtime deviation nor a change in available supply-chain trust information are outside the scope of this work.

PROVBIND does not assume that a valid signature, provenance attestation, or SBOM implies that the corresponding software is benign. Rather, its security objective is to determine whether an admitted workload remains consistent with its authenticated supply-chain evidence during execution and whether the trust underlying that evidence remains valid throughout the workload lifetime.

C. System Process

PROVBIND operates through six phases that carry authenticated software supply-chain evidence from artifact construction to continuous runtime verification. The process separates evidence establishment, admission, envelope construction, runtime verification, deviation analysis, and post-admission trust re-evaluation, as illustrated in Fig. 1.

Phase 1: Build and Supply-Chain Evidence Generation.: This phase establishes the authenticated supply-chain evidence used throughout PROVBIND. The Software Producer (SP) builds the container image, generates its software-composition and build-provenance evidence, binds them to the immutable image identity, and publishes the resulting authenticated artifact record to the Supply-Chain Repository (SCR).

Step 1: Container Image Construction. Given source repository S , Dockerfile F_D , dependency manifest M , and base image I_B , the SP invokes the CI pipeline to construct container image

$$I \leftarrow \text{Build}(S, F_D, M, I_B). \quad (1)$$

The resulting image is identified by its immutable OCI digest

$$d_I = \text{Digest}(\text{Manifest}(I)), \quad (2)$$

which serves as the common artifact identifier across admission, runtime verification, attribution, and trust re-evaluation. The image manifest M_I and configuration C_I retain the layer ordering, entrypoint, and other metadata required for subsequent envelope construction.

Step 2: Supply-Chain Evidence Generation. For image I , the SP generates SBOM B_I and provenance attestation P_I :

$$B_I \leftarrow \text{SBOMGen}(I), \quad P_I \leftarrow \text{ProvGen}(S, F_D, M, I_B, d_I). \quad (3)$$

B_I records the available software-component and dependency information, whereas P_I records the build context, including the source, builder identity, build parameters, and resulting artifact identity. Both evidence objects reference d_I , thereby associating their declarations with the same immutable container image.

Step 3: Evidence Binding and Validation. The SP constructs the supply-chain evidence bundle

$$\mathcal{B}_I = (d_I, M_I, C_I, B_I, P_I) \quad (4)$$

and verifies that the artifact identities referenced by the generated evidence are consistent with d_I . An inconsistent or incomplete binding is rejected before publication. This ensures that the image structure, software composition, and build provenance consumed by later phases refer to the same artifact.

Step 4: Signing and Transparency Recording. The SP authenticates the artifact using its authorized signing key maintained by the KMS:

$$\sigma_I \leftarrow \text{Sign}_{sk_{SP}}(d_I). \quad (5)$$

The signing event is recorded by the transparency service, which provides corresponding transparency evidence τ_I . PROVBIND therefore associates the image with the authenticated artifact record

$$\mathcal{A}_I = (\mathcal{B}_I, \sigma_I, \tau_I). \quad (6)$$

Step 5: Artifact and Evidence Publication. Finally, the SP publishes I and $\mathcal{A}_I$ to the SCR, indexed by d_I . The resulting record provides the artifact identity, manifest and configuration, SBOM, provenance, signature, and transparency evidence required by subsequent phases. Phase 2 retrieves this record and verifies that the image presented for Kubernetes admission is the authenticated artifact established in this phase.

Phase 2: Admission and Evidence Verification.: This phase establishes the verified artifact state from which PROVBIND constructs its runtime envelope. For each deployment request, the Admission and Envelope Manager (AEM) resolves the requested image, retrieves its authenticated artifact record from the Supply-Chain Repository (SCR), and verifies both the authenticity of the evidence and its binding to the image being admitted. Only evidence that passes all required checks is permitted to influence subsequent runtime verification.

Step 1: Image Resolution and Evidence Retrieval. For a workload requesting image reference r_I , the AEM first resolves the reference to the immutable image digest

$$d_I \leftarrow \text{Resolve}(r_I). \quad (7)$$

It then retrieves the authenticated artifact record

$$\mathcal{A}_I = (\mathcal{B}_I, \sigma_I, \tau_I), \quad (8)$$

where

$$\mathcal{B}_I = (d_I, M_I, C_I, B_I, P_I) \quad (9)$$

contains the image manifest M_I , image configuration C_I , SBOM B_I , and provenance attestation P_I established in Phase 1. Indexing retrieval by d_I ensures that mutable image tags cannot redirect the admission process to evidence belonging to another image version.

Step 2: Artifact Authentication. The AEM verifies the image signature against an authorized signer and validates the associated transparency evidence. Let

$$v_{\text{sig}} = \text{VerifySig}(d_I, \sigma_I, pk_{\text{SP}}) \quad (10)$$

denote signature validity, and

$$v_{\text{trans}} = \text{VerifyTrans}(d_I, \sigma_I, \tau_I) \quad (11)$$

denote successful verification of the corresponding transparency record. The AEM additionally checks whether the signer and signing key are acceptable under the current admission policy:

$$v_{\text{trust}} = \text{TrustCheck}(pk_{\text{SP}}, t), \quad (12)$$

where t denotes the current verification time. The authentication condition is therefore

$$v_{\text{auth}} = v_{\text{sig}} \wedge v_{\text{trans}} \wedge v_{\text{trust}}. \quad (13)$$

If $v_{\text{auth}} = 0$, admission terminates and the workload is rejected.

Step 3: Cross-Evidence Binding Verification. After successful authentication, the AEM verifies that all evidence objects refer to the same resolved artifact d_I . Specifically, it checks the consistency of the manifest and configuration with the resolved image, verifies that the provenance identifies d_I as the produced artifact, and confirms that the SBOM is associated with the same artifact record. We define

$$v_{\text{bind}} = v_M \wedge v_C \wedge v_B \wedge v_P, \quad (14)$$

where

$$v_M = \text{MatchManifest}(d_I, M_I), \quad (15)$$

$$v_C = \text{MatchConfig}(M_I, C_I), \quad (16)$$

$$v_B = \text{BindSBOM}(B_I, d_I), \quad (17)$$

$$v_P = \text{BindProv}(P_I, d_I). \quad (18)$$

This verification prevents otherwise valid evidence associated with a different image or artifact version from being reused during admission.

Step 4: Admission Decision and Verified-State Establishment. The final admission decision combines authentication and binding validity:

$$v_I = v_{\text{auth}} \wedge v_{\text{bind}}. \quad (19)$$

Accordingly,

$$\text{Decision}(I) = \begin{cases} \text{Admit}, & v_I = 1, \\ \text{Reject}, & v_I = 0. \end{cases} \quad (20)$$

For an admitted workload, the AEM establishes the verified artifact state

$$\mathcal{V}_I = (d_I, M_I, C_I, B_I, P_I), \quad (21)$$

which becomes the sole supply-chain input accepted by the Envelope Compiler. The corresponding verification context

$$\Gamma_I = (pk_{\text{SP}}, \sigma_I, \tau_I, t, v_I) \quad (22)$$

is retained for audit and subsequent trust re-evaluation.

Step 5: Verified-Evidence Handoff. Upon successful admission, the AEM forwards $\mathcal{V}_I$ to Phase 3 while retaining Γ_I as the authenticated trust context of the running image:

$$(\mathcal{V}_I, \Gamma_I) \xrightarrow{\text{Admit}} \text{EnvelopeCompiler}. \quad (23)$$

Only evidence contained in $\mathcal{V}_I$ may contribute to the runtime envelope. Phase 3 therefore transforms an authenticated and artifact-consistent supply-chain state into the provenance-bound runtime reference used during execution.

textbf{Phase 3: Provenance-Bound Runtime Envelope Compilation.} This phase transforms the verified artifact state $\mathcal{V}_I$ established in Phase 2 into the runtime state used by PROVBIND. The Admission and Envelope Manager (AEM) derives deterministic expectations from authenticated image, SBOM, and provenance evidence, separately derives properties not explicitly represented by such evidence, and constructs the behavioral, indexing, and provenance structures required for runtime verification and attribution.

Step 1: File and Image-Layer Derivation. From manifest M_I , the AEM obtains the ordered image layers

$$\mathcal{L}_I = (L_1, \dots, L_m) \quad (24)$$

and reconstructs the effective filesystem by applying layer replacement and whiteout semantics. The resulting file reference is

$$\mathcal{F}_I = \{(f, h_f, \ell_f, \mu_f)\}, \quad (25)$$

where f is an effective file, h_f its expected content hash, ℓ_f the layer containing its effective version, and μ_f its relevant metadata. The layer mapping

$$\Lambda_I(f) = \ell_f \quad (26)$$

preserves the image-layer origin of each effective file for subsequent runtime attribution.

Step 2: Package and Execution-Closure Derivation. The AEM parses SBOM B_I into the package-dependency graph

$$G_I^P = (\mathcal{P}_I, \mathcal{D}_I), \quad (27)$$

where $\mathcal{P}_I$ contains the identified packages and $\mathcal{D}_I$ their available dependency relationships. From the identified root components $\mathcal{P}_I^{\text{root}}$, it derives

$$\mathcal{P}_I^* = \text{Reach}(G_I^P, \mathcal{P}_I^{\text{root}}) \quad (28)$$

and assigns each reachable package p its minimum dependency depth

$$\delta_I(p) = \min_{r \in \mathcal{P}_I^{root}} \text{dist}_{G_I^P}(r, p). \quad (29)$$

Packages lacking sufficient dependency information are explicitly marked unresolved. Where ownership information is available, the compiler additionally establishes

$$\Phi_I : f \mapsto p. \quad (30)$$

Using configuration C_I and the effective filesystem $\mathcal{F}_I$, the AEM constructs the statically resolvable execution/load graph $G_I^L = (V_I^L, E_I^L)$. Starting from the declared entrypoint q_0 , it derives

$$\mathcal{Q}_I = \text{Reach}(G_I^L, q_0), \quad (31)$$

which defines the declaration-derived execution reference. Dynamically generated, interpreted, or runtime-installed code is not assumed to be exhaustively recoverable by this static closure.

Step 3: Supplementary Runtime and Behavioral Context.

The declaration-derived state may not completely specify the runtime capabilities expected by an application. PROVBIND therefore uses a pre-trained LightGBM classifier [21] only to supplement missing capability information. From the verified artifact state, the AEM constructs the feature vector

$$\mathbf{z}_I = \Omega_I(\mathcal{C}_I^{cfg}, \mathcal{P}_I^*, \mathcal{Q}_I, C_I), \quad (32)$$

where $\Omega_I(\cdot)$ extracts artifact and workload features from the container configuration, reachable packages, executable closure, and deployment context.

The pre-trained multi-label LightGBM model predicts candidate runtime capabilities as

$$\hat{\mathcal{C}}_I = \text{LGBM}_{cap}(\mathbf{z}_I). \quad (33)$$

The prediction cannot grant capabilities beyond those permitted by the effective Kubernetes security configuration. Hence,

$$\mathcal{C}_I = \hat{\mathcal{C}}_I \cap \mathcal{C}_I^{K8s}, \quad (34)$$

where $\mathcal{C}_I^{K8s}$ denotes the effective capability set permitted for the workload. Algorithm 1 summarizes this procedure. Essentially, the feature extractor Ω_I converts the verified artifact state, reachable package set, and execution closure into the fixed-dimensional feature vector used by the pre-trained LightGBM classifier. Features include artifact configuration, package/dependency indicators, executable and library characteristics, and deployment attributes available at admission time.

Unlike learned policy generation that treats predicted behavior as authoritative, PROVBIND uses LightGBM only to supplement capability information absent from authenticated evidence. Predictions are bounded by the effective Kubernetes capability set and retain an INFERRED provenance label, while declaration-derived capabilities remain AUTHENTICATED. This evidence-aware separation allows subsequent runtime

Algorithm 1 Evidence-Aware Capability Inference

Require: Verified state $\mathcal{V}_I$, package closure $\mathcal{P}_I^*$, execution closure $\mathcal{Q}_I$, declared capabilities $\mathcal{C}_I^{decl}$, effective Kubernetes capability set $\mathcal{C}_I^{K8s}$, trained model LGBM_{cap} , threshold θ_C

Ensure: Capability expectation $\mathcal{C}_I$ with provenance labels $\pi(\cdot)$

```

1:  $\mathbf{z}_I \leftarrow \Omega_I(\mathcal{V}_I, \mathcal{P}_I^*, \mathcal{Q}_I)$ 
2:  $\mathbf{p}_I \leftarrow \text{LGBM}_{cap}(\mathbf{z}_I)$ 
3:  $\hat{\mathcal{C}}_I^{inf} \leftarrow \{c \mid p_I(c) \geq \theta_C\}$ 
4:  $\hat{\mathcal{C}}_I^{inf} \leftarrow \hat{\mathcal{C}}_I^{inf} \cap \mathcal{C}_I^{K8s}$ 
5:  $\mathcal{C}_I^{decl} \leftarrow \mathcal{C}_I^{decl} \cap \mathcal{C}_I^{K8s}$ 
6:  $\mathcal{C}_I \leftarrow \mathcal{C}_I^{decl} \cup \hat{\mathcal{C}}_I^{inf}$ 
7: for all  $c \in \mathcal{C}_I$  do
8:   if  $c \in \mathcal{C}_I^{decl}$  then
9:      $\pi(c) \leftarrow \text{AUTHENTICATED}$ 
10:  else
11:     $\pi(c) \leftarrow \text{INFERRED}$ 
12:  end if
13: end for
14: return  $(\mathcal{C}_I, \pi)$ 

```

verification and scoring to account for the origin and confidence of each envelope constraint.

Step 4: Provenance-Bound Envelope and Runtime Indices.

The AEM compiles the deterministic runtime envelope

$$\mathcal{E}_I = (\mathcal{F}_I, G_I^P, \delta_I, \Phi_I, \mathcal{Q}_I, \mathcal{C}_I, \hat{\mathcal{N}}_I). \quad (35)$$

Each expectation $x \in \mathcal{E}_I$ retains its evidentiary origin

$$\pi(x) \in \{\text{AUTHENTICATED}, \text{INFERRED}\}, \quad (36)$$

preventing model-derived expectations from being treated as authenticated supply-chain claims.

For efficient online verification, the AEM further constructs

$$\mathcal{J}_I = (J_{\text{path}}, J_{\text{hash}}, J_{\text{layer}}, J_{\text{pkg}}, J_{\text{depth}}), \quad (37)$$

which supports direct lookup without traversing the complete envelope for each runtime event.

Step 5: Provenance Graph Construction and State Storage.

The AEM organizes the provenance relationships derived above into the typed graph maintained by the EPGS. As illustrated in Fig. 2, the graph connects the admitted image with its layers, packages, files, running container instances, and processes, providing the structural basis for package/dependency and image-layer attribution in Phase 5.

For image I , the EPGS maintains the typed property graph

$$G_I^{prov} = (V_I^{prov}, E_I^{prov}, \phi_V, \phi_E, \mathcal{A}_V), \quad (38)$$

where V_I^{prov} and E_I^{prov} are the vertex and directed-edge sets, ϕ_V and ϕ_E assign vertex and relationship types, and $\mathcal{A}_V$ associates vertices with their available provenance attributes. The supported vertex types are

$$\phi_V(v) \in \{\text{IMAGE}, \text{LAYER}, \text{PACKAGE}, \text{FILE}, \text{CONTAINER}, \text{PROCESS}\}, \quad (39)$$

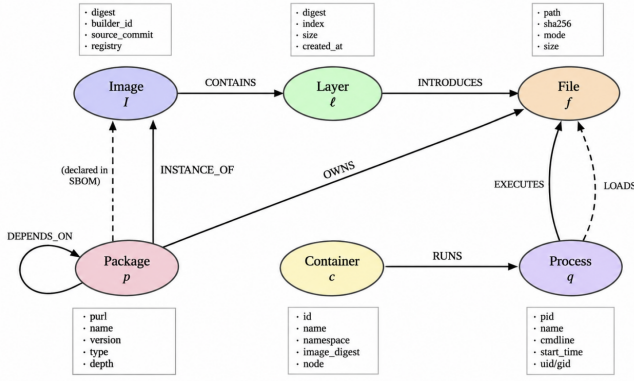


Fig. 2. PROVIND provenance graph model linking the admitted image to its software, image-layer, and runtime provenance. Dashed edges denote optional or derived relationships.

and the principal relationship types are

$$\phi_E(e) \in \{\text{CONTAINS, INTRODUCES, DEPENDS_ON, OWNS, INSTANCE_OF, RUNS, EXECUTES, LOADS}\}^{e_t = (c, d_I, q_t, \tau_t, o_t, a_t, \mathbf{z}_t, t)} \quad (40)$$

For layer ℓ , packages p_i, p_j , file f , container c , and process q , the principal relationships are instantiated as

$$I \xrightarrow{\text{CONTAINS}} \ell \iff \ell \in \mathcal{L}_I, \quad (41)$$

$$\ell \xrightarrow{\text{INTRODUCES}} f \iff \Lambda_I(f) = \ell, \quad (42)$$

$$p_i \xrightarrow{\text{DEPENDS_ON}} p_j \iff (p_i, p_j) \in \mathcal{D}_I, \quad (43)$$

$$p_i \xrightarrow{\text{OWNS}} f \iff \Phi_I(f) = p_i, \quad (44)$$

$$c \xrightarrow{\text{INSTANCE_OF}} I \iff \text{Image}(c) = d_I, \quad (45)$$

$$c \xrightarrow{\text{RUNS}} q \iff \text{Container}(q) = c, \quad (46)$$

$$q \xrightarrow{\text{EXECUTES}} f \iff \text{Exec}(q, f), \quad (47)$$

$$q \xrightarrow{\text{LOADS}} f \iff \text{Load}(q, f). \quad (48)$$

The first four relationships are compiled from the authenticated image and SBOM state, whereas container, process, execution, and load relationships are instantiated as the admitted workload executes. This distinction preserves the static supply-chain provenance while allowing runtime observations to be bound to the same artifact graph.

The attribute function $\mathcal{A}_V(v)$ retains the available type-specific metadata, including image digest and builder identity, layer index, package identity, version and dependency depth, file path and expected hash, container identity and namespace, and process identity and execution context. Attributes inherited from authenticated evidence retain their origin through $\pi(\cdot)$; unavailable relationships or attributes are marked unresolved rather than inferred.

Finally, the EPGS stores the compiled artifact state as

$$\text{EPGS}[d_I] \leftarrow (\mathcal{E}_I, \mathcal{M}_I^{\text{beh}}, \mathcal{J}_I, G_I^{\text{prov}}). \quad (49)$$

Here, $(\mathcal{E}_I, \mathcal{J}_I)$ supports deterministic runtime verification, $\mathcal{M}_I^{\text{beh}}$ supports in-process behavioral analysis, and G_I^{prov}

provides the typed provenance state used for runtime binding and attribution in Phases 4 and 5.

Phase 4: Runtime Observation and Dual-Path Verification. This phase continuously verifies whether an admitted workload remains consistent with the runtime state compiled in Phase 3. The Kubernetes Worker Node (KWN) collects security-relevant events through eBPF, while the Runtime Integrity Verifier (RIV) applies two complementary detection paths. First, each event is deterministically checked against the provenance-bound envelope $\mathcal{E}_I$. Second, envelope-conforming events from monitored processes are aggregated over observation windows and examined for anomalous in-process behavior using $\mathcal{M}_I^{\text{beh}}$.

Step 1: Runtime Event Collection and Artifact Binding. For each running container c , the eBPF collector observes process, file, capability, and network activities. Each observation is represented as

where d_I is the immutable image digest, q_t identifies the originating process, τ_t is the event type, o_t is the observed object, a_t is the operation, $\mathbf{z}_t$ contains relevant event context, and t is the observation time. Before verification, the RIV checks that the event belongs to the admitted artifact:

$$v_{\text{bind}}(e_t) = \text{BindCheck}(c, d_I). \quad (51)$$

An unknown or inconsistent binding is immediately reported as a binding failure and is never evaluated against another artifact's envelope.

Step 2: Cached Runtime-State Resolution. For $v_{\text{bind}}(e_t) = 1$, the RIV resolves the online state associated with d_I :

$$(\mathcal{E}_I, \mathcal{J}_I, \mathcal{M}_I^{\text{beh}}) \leftarrow \text{Cache}[d_I]. \quad (52)$$

On a cache miss, the required state is retrieved from the EPGS and inserted into the node-local cache. A Cuckoo filter provides a compact membership test before detailed indexed lookup. The provenance graph G_I^{prov} is deliberately excluded from this per-event path and is accessed only when attribution is required in Phase 5.

Step 3: Provenance-Bound Deterministic Verification. The event type τ_t selects the applicable envelope clause and index $J_{\tau_t} \subseteq \mathcal{J}_I$. The RIV evaluates

$$v_{\text{env}}(e_t) = \text{Verify}_{\tau_t}(e_t, \mathcal{E}_I, J_{\tau_t}) \in \{0, 1\}, \quad (53)$$

where $v_{\text{env}}(e_t) = 0$ denotes a direct contradiction with the compiled runtime reference. The deterministic deviation classes are

$$\mathcal{D}^{\text{det}} = \{D_{\text{exec}}, D_{\text{load}}, D_{\text{write}}, D_{\text{cap}}, D_{\text{net}}, D_{\text{hash}}\}, \quad (54)$$

representing undeclared execution, unexpected executable-code loading, protected-file modification, capability excess, network-egress inconsistency, and modified or relocated code, respectively.

For example, execution of file f is consistent only when both its execution membership and expected identity hold:

$$V_{\text{exec}}(f) = \mathbf{1}[f \in \mathcal{Q}_I] \wedge \mathbf{1}[H(f) = h_I(f)], \quad (55)$$

where $h_I(f)$ is obtained through the indexed file envelope. Similarly, an observed capability c_p satisfies

$$V_{\text{cap}}(c_p) = \mathbf{1}[c_p \in \mathcal{C}_I]. \quad (56)$$

Runtime-created mutable objects are not classified as contradictions solely because they are absent from $\mathcal{F}_I$, provided that they do not modify a protected image-declared object or violate another envelope clause.

Step 4: In-Process Behavioral Anomaly Detection. An event satisfying $v_{\text{env}}(e_t) = 1$ enters the behavioral path only when its originating process $q_t \in \mathcal{Q}_I^{\text{beh}}$. For each monitored process q , the RIV maintains a process-specific observation window W_q containing only envelope-conforming events generated by that process. The corresponding behavioral feature vector is

$$\mathbf{x}_{q,W} = \Psi_I(\{e_t \mid q_t = q, t \in W_q, v_{\text{env}}(e_t) = 1\}), \quad (57)$$

where Ψ_I is the process-level feature extractor defined by $\mathcal{M}_I^{\text{beh}}$.

The trained Isolation Forest model [22] evaluates the aggregated process behavior:

$$s_A(q, W) = \text{IF}_I(\mathbf{x}_{q,W}). \quad (58)$$

The score is normalized such that larger values indicate stronger anomalousness. The behavioral decision is therefore

$$a_{\text{beh}}(q, W) = \mathbf{1}[s_A(q, W) \geq \theta_I^A], \quad (59)$$

where θ_I^A is the anomaly threshold associated with the behavioral model.

Algorithm 2 summarizes the online procedure. Unlike standalone anomaly detection, PROVBIND places behavioral analysis behind deterministic provenance-bound verification. Only envelope-conforming events enter the process-specific behavioral window, while explicit contradictions are handled directly by Step 3. The two paths therefore complement each other: deterministic verification detects envelope violations, whereas Isolation Forest identifies anomalous patterns within otherwise permitted behavior. Each behavioral finding remains bound to the admitted image d_I and process q , enabling provenance-aware attribution in Phase 5. Thus, the novelty lies in the provenance-gated dual-path verification design rather than the Isolation Forest model itself.

Thus, the behavioral path operates only on processes whose individual runtime events remain consistent with the deterministic envelope. Its purpose is to identify abnormal temporal or interaction patterns within otherwise permitted execution, thereby complementing rather than replacing provenance-bound verification. **Step 5: Detection Outcome and Deviation Generation.** The RIV distinguishes event-level deterministic deviations from window-level behavioral anomalies.

Algorithm 2 Provenance-Gated Behavioral Anomaly Detection

Require: $e_t, d_I, \mathcal{M}_I^{\text{beh}} = (\text{IF}_I, \Psi_I, g_I, \theta_I^A, W_I, \mathcal{Q}_I^{\text{beh}})$

Ensure: $D_{q,W}^{\text{beh}}$ or $\emptyset$

```

1:  $q \leftarrow \text{Process}(e_t)$ 
2: if  $v_{\text{env}}(e_t) = 0 \vee q \notin \mathcal{Q}_I^{\text{beh}}$  then
3:   return  $\emptyset$ 
4: end if
5:  $W_q \leftarrow \text{UpdateWindow}(W_q, e_t, W_I)$ 
6: if  $\neg \text{WindowReady}(W_q)$  then
7:   return  $\emptyset$ 
8: end if
9:  $\mathbf{x}_{q,W} \leftarrow \Psi_I(W_q)$ 
10:  $s_A \leftarrow \text{IF}_I(\mathbf{x}_{q,W})$ 
11:  $\bar{s}_A \leftarrow g_I(s_A)$ 
12: if  $\bar{s}_A \geq \theta_I^A$  then
13:    $D_{q,W}^{\text{beh}} \leftarrow (d_I, q, W_q, \bar{s}_A, \mathbf{x}_{q,W}, t_W)$ 
14:   return  $D_{q,W}^{\text{beh}}$ 
15: end if
16: return  $\emptyset$ 
```

A binding or envelope contradiction is reported immediately as

$$D_t^{\text{det}} = (d_I, e_t, \tau_D, x_D, \pi(x_D), t), \quad (60)$$

where τ_D identifies the deviation class, x_D denotes the violated envelope clause, and $\pi(x_D)$ preserves whether that expectation originates from authenticated or inferred evidence.

For an envelope-conforming monitored process, the behavioral decision is made when its observation window completes. If $a_{\text{beh}}(q, W) = 1$, the RIV generates

$$D_{q,W}^{\text{beh}} = (d_I, q, W, s_A(q, W), \mathbf{x}_{q,W}, t_W), \quad (61)$$

where t_W is the window-completion time. Otherwise, the behavior is classified as conforming. Both D_t^{det} and $D_{q,W}^{\text{beh}}$ are forwarded to Phase 5 for provenance-aware scoring, attribution, and alert generation.

Phase 5: Provenance-Aware Scoring and Attribution.:

This phase transforms detections produced in Phase 4 into explainable and provenance-aware alerts. The Runtime Integrity Verifier (RIV) resolves each deterministic deviation or behavioral anomaly against the typed provenance graph maintained by the EPGS, evaluates its security significance, traces its package/dependency and image-layer origins, and generates an attribution record for investigation and audit.

Step 1: Detection and Provenance-Context Resolution. Let

$$D \in \{D_t^{\text{det}}, D_{q,W}^{\text{beh}}\} \quad (62)$$

denote a detection received from Phase 4. Using its immutable image digest d_I , the RIV retrieves

$$G_I^{\text{prov}} \leftarrow \text{EPGS}[d_I]. \quad (63)$$

For D_t^{det} , the implicated runtime object o_D is resolved directly from event e_t and its violated envelope clause x_D . For $D_{q,W}^{\text{beh}}$, o_D is the monitored process q , from which the

RIV resolves the executable and software objects contributing to the anomalous window. Using the mappings established in Phase 3, the resulting context is

$$\mathcal{C}_D = (o_D, f_D, p_D, \delta_D, \ell_D), \quad (64)$$

where f_D is the associated file, p_D its owning package when known, $\delta_D = \delta_I(p_D)$ its dependency depth, and $\ell_D = \Lambda_I(f_D)$ its effective image layer. Missing relationships remain explicitly unresolved rather than being inferred.

Step 2: Detection-Aware Severity Scoring. PROVBIND preserves the different evidentiary meanings of deterministic and behavioral detections. For a deterministic deviation, the prioritization score is

$$s_{det}(D) = w_\tau s_\tau(\tau_D) + w_\pi s_\pi(\pi(x_D)) + w_c s_c(\mathcal{C}_D), \quad (65)$$

where s_τ reflects the security significance of the contradiction class, s_π reflects the evidentiary origin of the violated expectation, and s_c captures configurable provenance context, including dependency position when relevant. In particular, a contradiction against authenticated evidence carries stronger evidentiary weight than one involving an inferred expectation:

$$s_\pi(\text{AUTHENTICATED}) > s_\pi(\text{INFERRED}). \quad (66)$$

For an in-process behavioral anomaly, no envelope contradiction is assumed. Its prioritization score is instead

$$s_{beh}(D) = w_A \bar{s}_A(q, W) + w_c s_c(\mathcal{C}_D), \quad (67)$$

where $\bar{s}_A(q, W)$ is the normalized Isolation Forest anomaly score. The resulting score

$$\hat{S}(D) \in [0, 1] \quad (68)$$

is used only for alert prioritization; it does not alter the detection decision made in Phase 4.

Step 3: Dual-Path Supply-Chain Provenance Attribution. Starting from the implicated object, the RIV traverses the typed relationships of G_I^{prov} to recover two complementary provenance views. The package/dependency path is

$$\mathcal{P}_D^{pkg} = \text{Trace}_{pkg}(G_I^{prov}, f_D), \quad (69)$$

which, when fully resolvable, has the form

$$p_{\text{root}} \xrightarrow{\text{DEPENDS_ON}^*} p_D \xrightarrow{\text{OWNS}} f_D. \quad (70)$$

Here, DEPENDS_ON^* denotes a zero-or-more-hop dependency path and exposes the dependency position through which p_D is introduced.

Independently, the image-layer path is

$$\mathcal{P}_D^{layer} = \text{Trace}_{layer}(G_I^{prov}, f_D, d_I), \quad (71)$$

with the form

$$I \xrightarrow{\text{CONTAINS}} \ell_D \xrightarrow{\text{INTRODUCES}} f_D. \quad (72)$$

For a behavioral anomaly, the runtime-process relation additionally provides

$$q \xrightarrow{\text{EXECUTES}} f_D, \quad (73)$$

or the corresponding LOADS relation when the anomalous behavior is associated with loaded executable content.

The complete attribution result is therefore

$$\mathcal{P}_D = (\mathcal{P}_D^{pkg}, \mathcal{P}_D^{layer}), \quad (74)$$

with the process relation included when applicable. This separates software-dependency origin from image-layer origin rather than assuming that packages and layers form a single linear provenance chain.

Step 4: Attribution Record and Explainable Alert. The RIV constructs the unified attribution record

$$A_D = (D, \hat{S}(D), \mathcal{C}_D, \mathcal{P}_D, \eta_D), \quad (75)$$

where η_D preserves detection-specific evidence. For a deterministic deviation,

$$\eta_D^{det} = (\tau_D, x_D, \pi(x_D)), \quad (76)$$

whereas for a behavioral anomaly,

$$\eta_D^{beh} = (q, W, \bar{s}_A(q, W)). \quad (77)$$

Accordingly, a deterministic alert identifies the violated envelope clause and whether it originates from authenticated or inferred evidence. A behavioral alert instead identifies the anomalous process, its behavioral score, and its resolved provenance context without claiming an envelope contradiction. The explanation therefore follows

$$\text{Alert}(D) : D \rightarrow o_D \rightarrow \{\mathcal{P}_D^{pkg}, \mathcal{P}_D^{layer}\} \rightarrow d_I. \quad (78)$$

Step 5: Tamper-Evident Violation Logging. Finally, each attribution record is appended to the ordered violation history. For the k th record, the RIV computes

$$H_k = H(H_{k-1} \parallel H(A_D)), \quad (79)$$

where H_{k-1} commits to the preceding history. The record

$$\mathcal{R}_k = (A_D, H_k) \quad (80)$$

is stored in the violation log. The hash chain provides an ordered, tamper-evident history under the assumed protection of the RIV and EPGS, supporting subsequent investigation and audit.

Phase 6: Continuous Supply-Chain Trust Re-evaluation.

This phase determines whether the supply-chain trust conditions that justified admission remain valid throughout workload execution. Unlike Phase 4, which verifies observed runtime behavior, the Trust Re-evaluator (TR) independently reassesses the authenticated evidence bound to the admitted artifact. Consequently, PROVBIND can identify a loss of supply-chain trust even when the workload remains consistent with its runtime envelope.

Step 1: Trust-State Initialization and Scheduling. For each admitted image I , the TR obtains the admission context

Γ_I established in Phase 2 together with the corresponding provenance and component evidence:

$$\Gamma_I = (pk_{SP}, \sigma_I, \tau_I, t_0, v_I). \quad (81)$$

All subsequent re-evaluation is indexed by the immutable artifact digest d_I , ensuring that trust updates are applied to the same image instance whose evidence was verified at admission.

Let trust re-evaluation occur at

$$t_r = t_0 + r\Delta_R, \quad r = 1, 2, \dots, \quad (82)$$

where Δ_R is the configured re-evaluation interval. The time-dependent trust state is

$$\mathcal{T}_I(t) = (T_I^{key}(t), T_I^{trans}(t), T_I^{builder}(t), T_I^{comp}(t)), \quad (83)$$

representing the current validity of the signing key, transparency evidence, builder identity, and component-level trust information. Since Phase 2 admits only successfully verified artifacts,

$$\text{Trusted}_I(t_0) = 1. \quad (84)$$

Step 2: Artifact-Bound Trust Re-evaluation. At each t_r , the TR reassesses the trust evidence associated with d_I . The individual trust predicates are

$$T_I^{key}(t_r) = \text{KeyCheck}(pk_{SP}, t_r), \quad (85)$$

$$T_I^{trans}(t_r) = \text{TransparencyCheck}(d_I, \sigma_I, \tau_I, t_r), \quad (86)$$

$$T_I^{builder}(t_r) = \text{BuilderCheck}(P_I, d_I, t_r), \quad (87)$$

$$T_I^{comp}(t_r) = \text{ComponentCheck}(B_I, d_I, t_r). \quad (88)$$

These predicates respectively determine whether the signing key remains acceptable, the transparency evidence remains valid for the admitted artifact, the builder identified by P_I remains acceptable, and component-level intelligence associated with B_I triggers a configured trust-policy condition.

The current artifact trust decision is

$$\text{Trusted}_I(t_r) = T_I^{key}(t_r) \wedge T_I^{trans}(t_r) \wedge T_I^{builder}(t_r) \wedge T_I^{comp}(t_r). \quad (89)$$

The interpretation of each predicate is policy controlled. In particular, discovery of a software vulnerability does not automatically imply trust withdrawal unless it satisfies the configured trust policy.

Step 3: Trust-Transition Detection and Cause Resolution.

Rather than repeatedly alerting on an already untrusted artifact, the TR detects changes in trust state. A trust-withdrawal transition is defined as

$$\Delta_I^-(t_r) = 1 [\text{Trusted}_I(t_{r-1}) = 1 \wedge \text{Trusted}_I(t_r) = 0]. \quad (90)$$

When $\Delta_I^-(t_r) = 1$, the failed trust conditions are identified as

$$\chi_I(t_r) = \{x \mid T_I^x(t_r) = 0\}, \quad x \in \{\text{key, trans, builder, comp}\}. \quad (91)$$

For a component-related failure, the TR additionally resolves the affected component through G_I^{prov} to recover its

available package/dependency and image-layer provenance. The resulting trust-transition record is

$$D_I^{trust} = (d_I, t_r, \mathcal{T}_I(t_{r-1}), \mathcal{T}_I(t_r), \chi_I(t_r), \mathcal{C}_I^{trust}), \quad (92)$$

where $\mathcal{C}_I^{trust}$ contains the available signer, builder, package, dependency, and layer context associated with the changed trust condition.

Step 4: Trust-State Update and Explainable Alert. After each re-evaluation, the EPGS updates the current trust state:

$$\text{EPGS}[d_I].\text{Trust} \leftarrow (\mathcal{T}_I(t_r), \text{Trusted}_I(t_r), t_r). \quad (93)$$

If $\Delta_I^-(t_r) = 1$, the TR emits the explainable trust alert

$$A_I^{trust} = (d_I, t_r, \chi_I(t_r), \mathcal{C}_I^{trust}), \quad (94)$$

which identifies both the affected artifact and the trust condition responsible for the transition. If the artifact remains untrusted at a later re-evaluation without a new state transition, its stored trust state is updated without generating a duplicate withdrawal alert.

Step 5: Post-Admission Trust Continuity. PROVBIND maintains runtime consistency and supply-chain trust as orthogonal post-admission states. Let

$$\mathcal{R}_I(t) \in \{\text{CONFORMING}, \text{DEVIATING}\} \quad (95)$$

denote the runtime state derived from Phase 4, while $\text{Trusted}_I(t)$ represents the independently re-evaluated supply-chain trust state. The joint post-admission state is therefore

$$\mathcal{S}_I(t) = (\mathcal{R}_I(t), \text{Trusted}_I(t)). \quad (96)$$

Accordingly, runtime deviation does not require prior withdrawal of supply-chain trust, and supply-chain trust may be withdrawn even when the workload remains runtime-conforming. This separation enables PROVBIND to preserve both behavioral and evidentiary continuity beyond the initial admission decision.

IV. EVALUATION PLAN

Detection quality is assessed against ground-truth labels with a supply chain violation as the positive class, reporting precision, recall, F_1 , false positive rate, and accuracy. False positive rate is reported alongside the F -measure rather than folded into it, because the central claim concerns precision specifically: verification against a signed specification should eliminate the class of false alarms arising when a learned baseline encounters legitimate but infrequent behaviour.

Published corpora supply runtime telemetry and chain-level ground truth but were assembled to evaluate detectors consuming telemetry alone, and ship without the signed provenance and SBOM documents this framework requires; the majority of their scenarios are additionally not containerised [?]. The attack semantics are therefore extracted and re-instantiated within an attested container testbed, and this reconstruction is reported explicitly as part of the methodology.

The framework is compared against a rule-based runtime monitor, an independently trained learned-anomaly detector,

and admission-time signature verification alone. Cost is measured on equal footing with accuracy: per-event verification latency, node-level CPU and memory overhead, kernel ring buffer drop rate under burst load, and compilation and indexing time per image. Overhead is reported relative to existing runtime collection rather than to an unmonitored host, since the kernel collection cost is inherited rather than introduced.

REFERENCES

- [1] M. Mounesan, H. Siadati, and S. Jafarikhah, "Exploring the threat of software supply chain attacks on containerized applications," in *Proc. 16th Int. Conf. Security of Information and Networks (SIN)*, 2023, pp. 1–8.
- [2] P. Ladisa, H. Plate, M. Martinez, and O. Barais, "SoK: Taxonomy of attacks on open-source software supply chains," in *Proc. IEEE Symp. Security and Privacy (SP)*, 2023, pp. 1509–1526.
- [3] C. Okafor, T. R. Schorlemmer, S. Torres-Arias, and J. C. Davis, "SoK: Analysis of software supply chain security by establishing secure design properties," in *Proc. ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses (SCORED)*, 2022, pp. 15–24.
- [4] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappel, "in-toto: Providing farm-to-table guarantees for bits and bytes," in *Proc. 28th USENIX Security Symp.*, 2019.
- [5] Open Source Security Foundation, "Supply-chain levels for software artifacts (SLSA), version 1.0 specification," OpenSSF, 2023.
- [6] Z. Newman, J. S. Meyers, and S. Torres-Arias, "Sigstore: Software signing for everybody," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2022, pp. 2353–2367.
- [7] S. Li, F. Dong, X. Xiao, H. Wang, F. Shao, J. Chen, Y. Guo, X. Chen, and D. Li, "NodLink: An online system for fine-grained APT attack detection and investigation," in *Proc. Network and Distributed System Security Symp. (NDSS)*, 2024.
- [8] B. Jiang et al., "ORTHRUS: Achieving high quality of attribution in provenance-based intrusion detection systems," in *Proc. 34th USENIX Security Symp.*, 2025.
- [9] T. Bilot, B. Jiang, Z. Li, N. El Madhoun, K. Al Agha, A. Zouaoui, and T. Pasquier, "Sometimes simpler is better: A comprehensive analysis of state-of-the-art provenance-based intrusion detection systems," in *Proc. 34th USENIX Security Symp.*, 2025.
- [10] T. R. Schorlemmer, K. G. Kalu, L. Chigges, K. M. Ko, E. A. Ishgair, S. Bagchi, S. Torres-Arias, and J. C. Davis, "Signing in four public software package registries: Quantity, quality, and influencing factors," in *Proc. IEEE Symp. Security and Privacy (SP)*, 2024, pp. 1160–1178.
- [11] B. Xia, T. Bi, Z. Xing, Q. Lu, and L. Zhu, "An empirical study on software bill of materials: Where we stand and the road ahead," in *Proc. IEEE/ACM 45th Int. Conf. Software Engineering (ICSE)*, 2023, pp. 2630–2642.
- [12] N. Zahan, E. Lin, M. Tamanna, W. Enck, and L. Williams, "Software bills of materials are required. Are we there yet?," *IEEE Security Privacy*, vol. 21, no. 2, pp. 82–88, 2023.
- [13] S. Yu, W. Song, X. Hu, and H. Yin, "On the correctness of metadata-based SBOM generation: A differential analysis approach," in *Proc. 54th Annu. IEEE/IFIP Int. Conf. Dependable Systems and Networks (DSN)*, 2024, pp. 29–36.
- [14] S. Ghavamnia, T. Palit, A. Mishra, and M. Polychronakis, "Confine: Automated system call policy generation for container attack surface reduction," in *Proc. Int. Symp. Research in Attacks, Intrusions and Defenses (RAID)*, 2020, pp. 443–458.
- [15] Y. Yang, B. B. Kang, and J. Nam, "Optimus: Association-based dynamic system call filtering for container attack surface reduction," *J. Cloud Computing*, vol. 13, no. 1, Art. no. 71, 2024.
- [16] M. S. Haq, T. D. Nguyen, A. Ş. Tosun, F. Vollmer, T. Korkmaz, and A.-R. Sadeghi, "SoK: A comprehensive analysis and evaluation of Docker container attack and defense mechanisms," in *Proc. IEEE Symp. Security and Privacy (SP)*, 2024, pp. 4573–4590.
- [17] S. Ryu et al., "Hybrid runtime detection of malicious containers using eBPF," *Computers, Materials Continua*, 2026.
- [18] I. N. Basuki and Y. Asnar, "Virtual firewall implementation with eBPF for distributed denial of service attacks in cloud-native environments," in *Proc. IEEE Int. Conf. Data and Software Engineering (ICoDSE)*, Batam, Indonesia, 2025, pp. 1–6, doi: 10.1109/ICoDSE68111.2025.11351565.
- [19] A. Goyal, X. Han, G. Wang, and A. Bates, "Sometimes, you aren't what you do: Mimicry attacks against provenance graph host intrusion detection systems," in *Proc. Network and Distributed System Security Symp. (NDSS)*, 2023.
- [20] T. Zheng et al., "KubeSec: Automatic detection of takeover risks introduced by third-party apps in the Kubernetes ecosystem," *IEEE Trans. Dependable Secure Comput.*, vol. 23, no. 3, pp. 7500–7513, May–Jun. 2026, doi: 10.1109/TDSC.2026.3673764.
- [21] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017.
- [22] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, 2008, pp. 413–422.
- [23] G. Yoon, J. Shin, J.-S. Kim, S. Kim, J. Jeong, and Y.-H. Choi, "Container-specific service mesh-based system for mitigating lateral movement attacks," *IEEE Trans. Cloud Comput.*, vol. 14, no. 1, pp. 92–109, Jan.–Mar. 2026, doi: 10.1109/TCC.2026.3657486.
- [24] S. Zehra, H. J. Syed, F. Samad, and U. Faseeha, "DeSFAM: An adaptive eBPF and AI-driven framework for securing cloud containers in real time," *IEEE Access*, vol. 13, pp. 139203–139224, 2025, doi: 10.1109/ACCESS.2025.3592192.
- [25] S. Bu, M. Jin, J. Wang, Y. Xie, and L. Zhang, "SST-LOF: Container anomaly detection method based on singular spectrum transformation and local outlier factor," *IEEE Trans. Cloud Comput.*, vol. 13, no. 1, pp. 130–147, Jan.–Mar. 2025, doi: 10.1109/TCC.2024.3514297.
- [26] W. Yuping, "A multi-level software supply chain security defense system for cloud-native and high-performance computing environments," in *Proc. Int. Conf. Frontiers of Information Technology (FIT)*, Islamabad, Pakistan, 2025, pp. 1–6, doi: 10.1109/FIT67061.2025.11333619.
- [27] N. Anjum et al., "Uncovering software supply chains vulnerability: A review of attack vectors, stakeholders, and regulatory frameworks," in *Proc. IEEE 47th Annu. Computers, Software, and Applications Conf. (COMPSAC)*, Torino, Italy, 2023, pp. 1816–1821, doi: 10.1109/COMP-SAC57700.2023.00281.
- [28] T. Tariq, W.-C. Kuo, K. Mahmood, S. Shamshad, A. K. Das, and M. J. F. Alenazi, "A lightweight authentication protocol for RFID-assisted supply chain management system," *IEEE Internet Things J.*, vol. 12, no. 9, pp. 12845–12852, May 2025, doi: 10.1109/JIOT.2024.3523390.
- [29] C. Luo, J. Gan, X. Li, Z. Pan, J. Tang, and Z. Tian, "Your botnet is his botnet? A deep dive into the supply chain attack against cyber-arm industry," *IEEE Trans. Netw.*, vol. 33, no. 5, pp. 2319–2335, Oct. 2025, doi: 10.1109/TON.2025.3566903.
- [30] S. Kalafatis, A. Villios, A. Garyfallou, N. Papageorgopoulos, and G. Ledakis, "C-Shield: LLM-enhanced cybersecurity platform for intrusion detection in Kubernetes clusters," in *Proc. 29th Conf. Innovation in Clouds, Internet and Networks (ICIN)*, Athens, Greece, 2026, pp. 1–3, doi: 10.1109/ICIN69025.2026.11481833.